SYSTEM DOCUMENTATION · BLN-C · COMPLETE REFERENCE

web app diagrams

Data Flow · Process Flow · UML Activity

Use Case · Systems Architecture · ERD

Database schema for bln-c web application

Supabase / PostgreSQL backend

01

DATA FLOW DIAGRAM

02

PROCESS FLOW DIAGRAM

03

UML ACTIVITY DIAGRAM

04

SYSTEMS FLOWCHART

05

USE CASE DIAGRAM

06

CONTEXT DIAGRAM

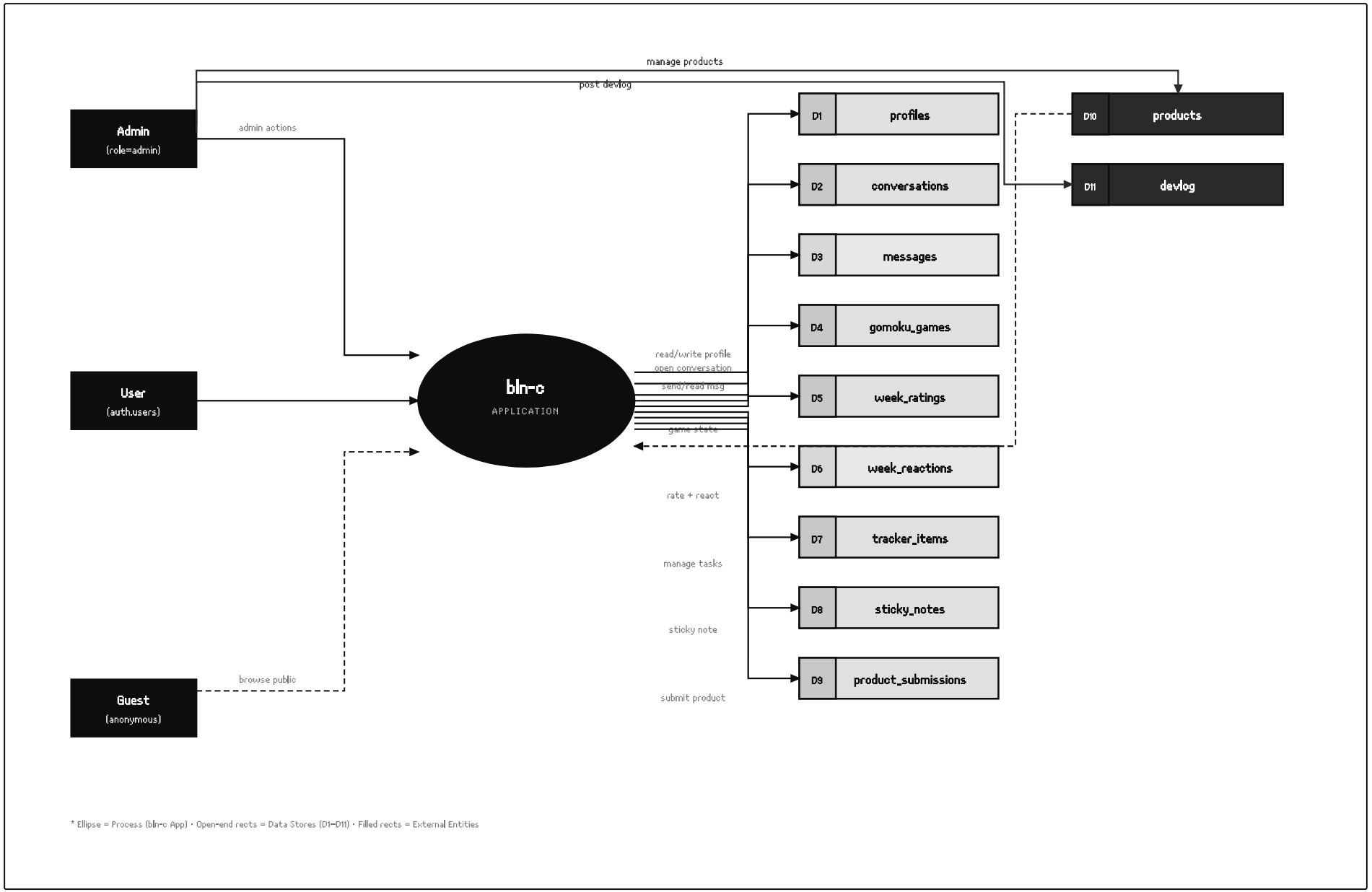
07

ENTITY-RELATIONSHIP

01

LEVEL 1 · DATA FLOW DIAGRAM

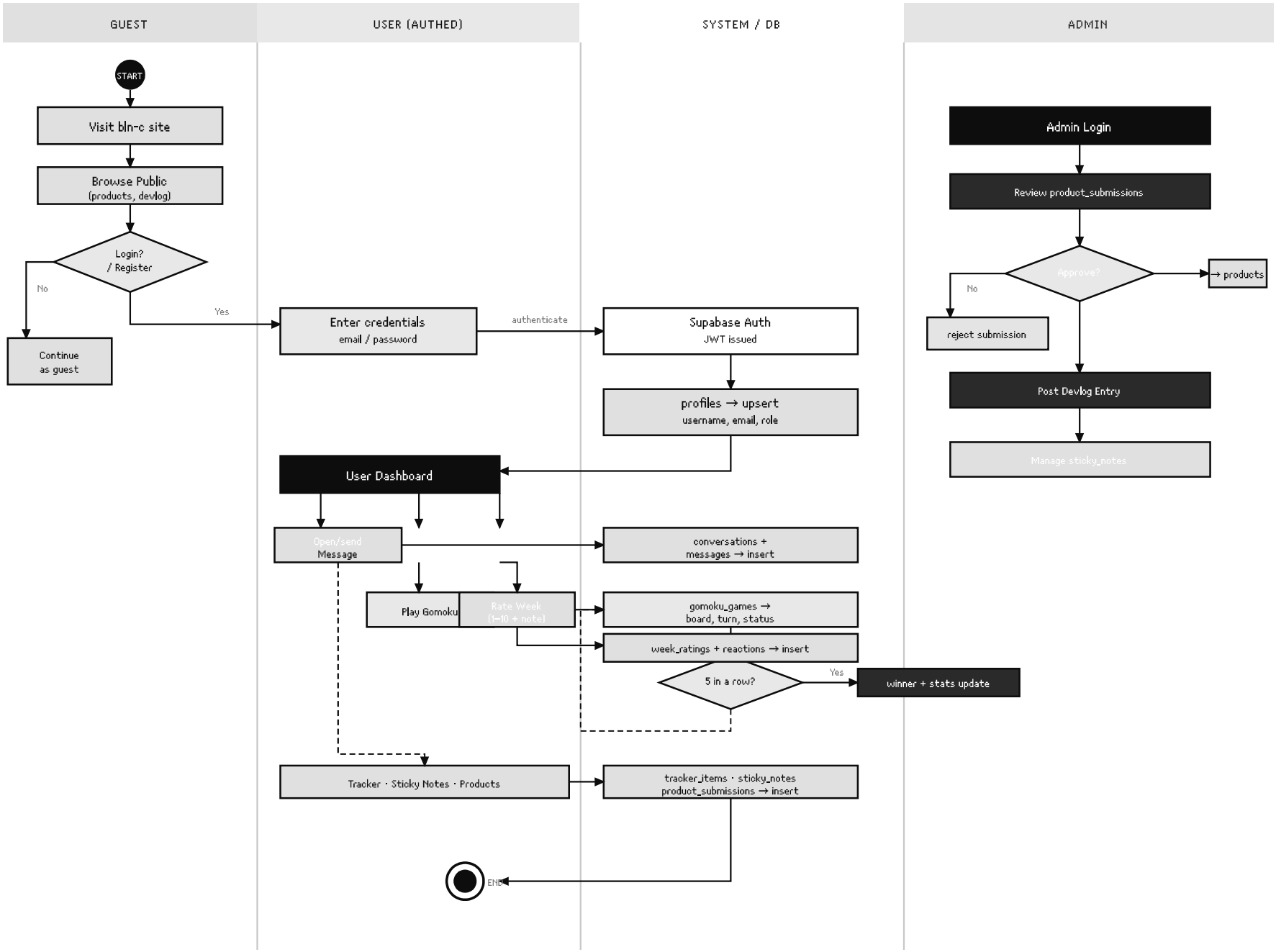
bln-c — Data Flow Diagram

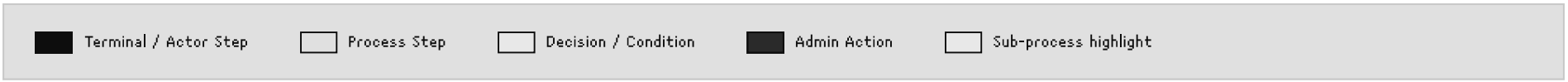

manage products
post devlog
Admin
(role=admin)
admin actions
User
(auth.users)
Guest
(anonymous)
browse public
bln-c
APPLICATION
read/write profile
open conversation
send/read msg
game state
rate + react
manage tasks
sticky note
submit product
D1 profiles
D2 conversations
D3 messages
D4 gomoku_games
D5 week_ratings
D6 week_reactions
D7 tracker_items
D8 sticky_notes
D9 product_submissions
D10 products
D11 devlog
* Ellipse = Process (bln-c App) · Open-end rects = Data Stores (D1–D11) · Filled rects = External Entities
External Entity (User/Admin/Guest)
Data Store (table)
Related Data Store (child table)
Admin-only Store
——→ Data flow
- - → Read-only / guest flow

02

PROCESS FLOW DIAGRAM

bln-c — Core Process Flows

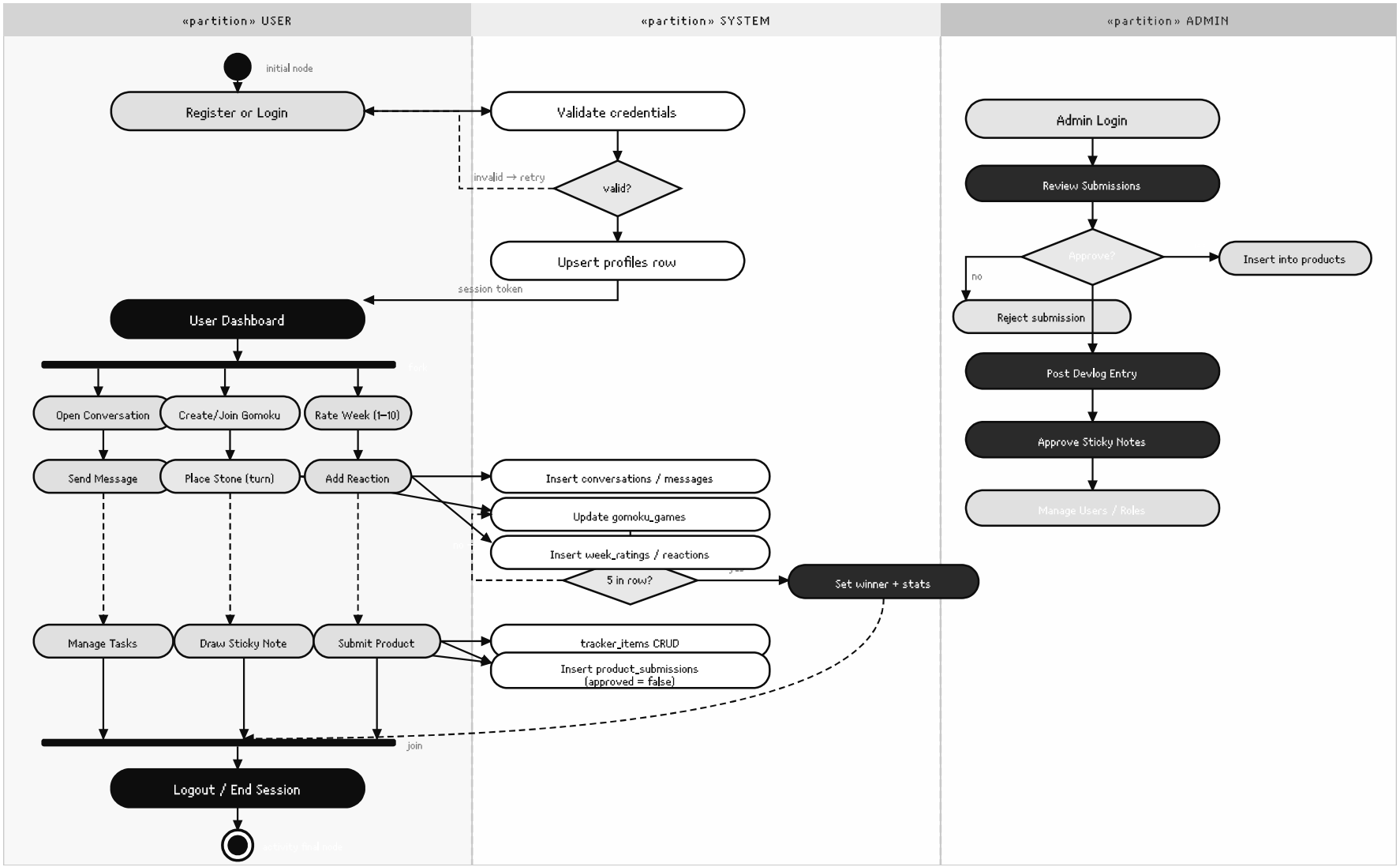

GUEST
USER (AUTHED)
SYSTEM / DB
ADMIN
START
Visit bln-c site
Browse Public
(products, devlog)
Login?
/ Register
No
Yes
Continue
as guest
Enter credentials
email / password
authenticate
Supabase Auth
JWT issued
profiles → upsert
username, email, role
User Dashboard
Open/send
Message
conversations +
messages → insert
Play Gomoku
Rate Week
(1–10 + note)
gomoku_games →
board, turn, status
week_ratings + reactions → insert
5 in a row?
Yes
winner + stats update
Tracker · Sticky Notes · Products
tracker_items · sticky_notes
product_submissions → insert
END
Admin Login
Review product_submissions
Approve?
→ products
No
reject submission
Post Devlog Entry
Manage sticky_notes


Terminal / Actor Step
Process Step
Decision / Condition
Admin Action
Sub-process highlight

03

UML 2.5 · ACTIVITY DIAGRAM

bln-c — User Activity (Full Lifecycle)



Initial Node

Activity Final Node

Fork / Join

User Action

System Action

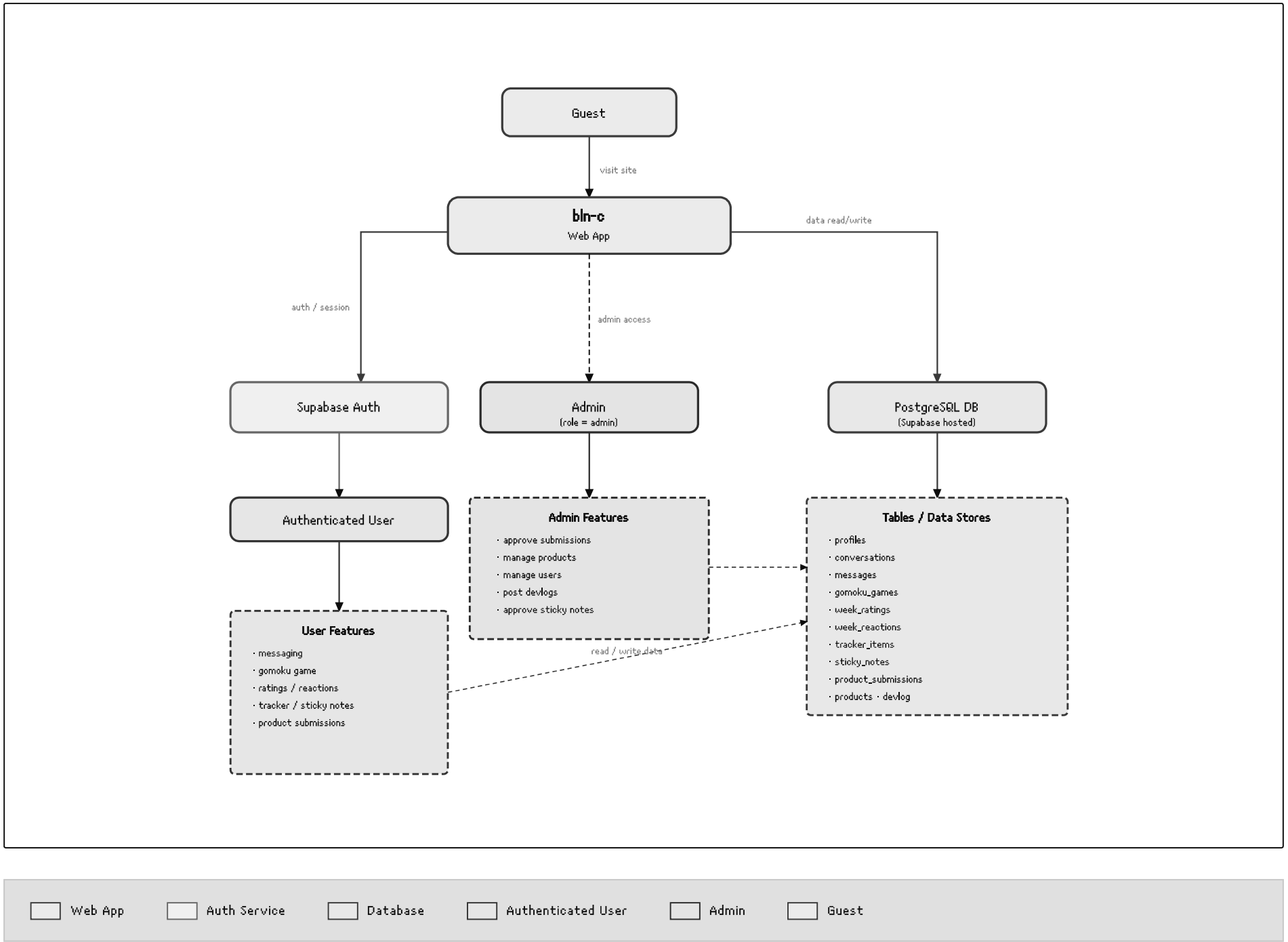
Admin Action

Decision Node

04

SYSTEMS FLOWCHART

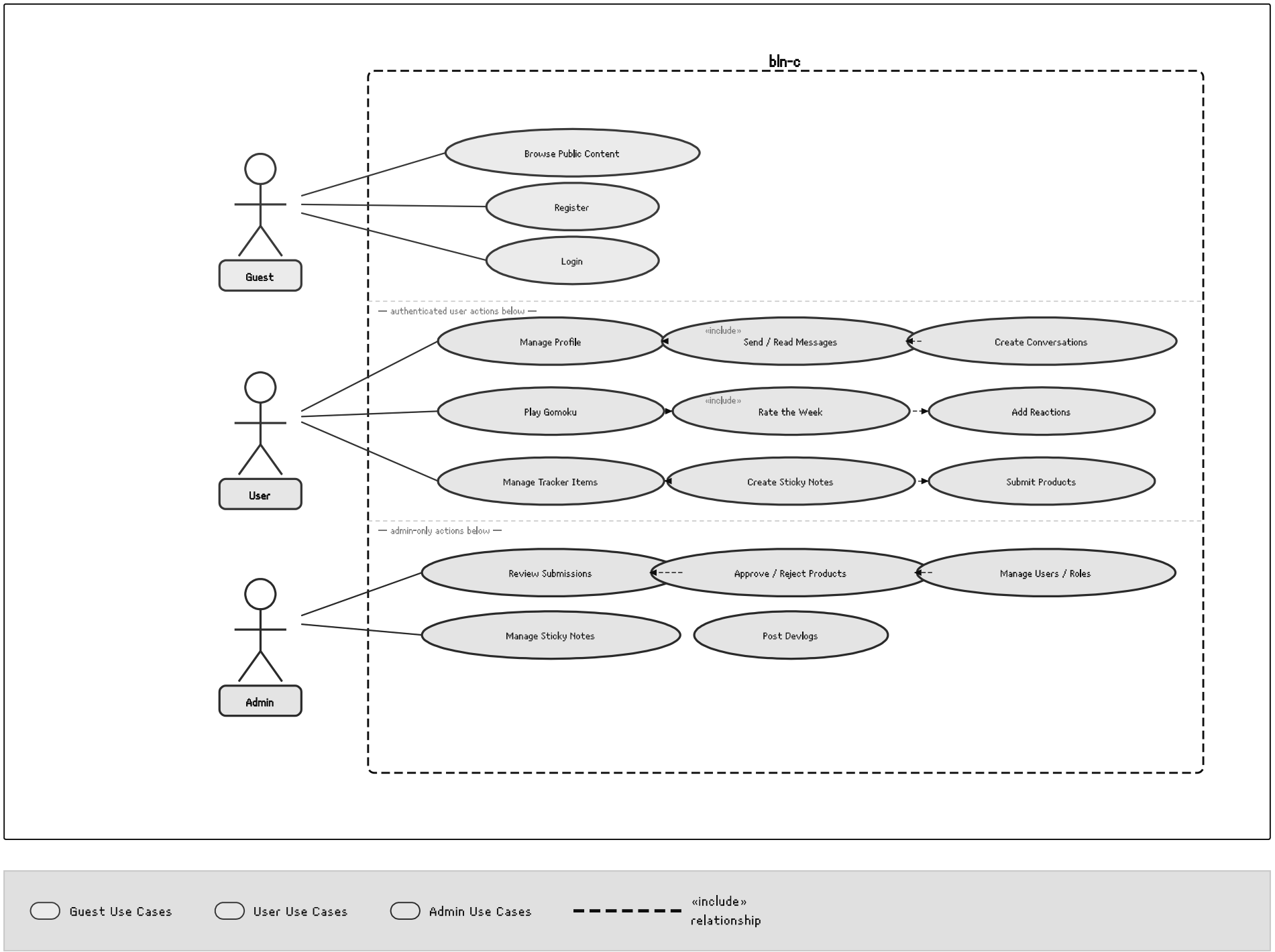
bln-c — Systems Architecture Flowchart


Guest
visit site
bln-c
Web App
data read/write
auth / session
admin access
Supabase Auth
Admin
(role = admin)
PostgreSQL DB
(Supabase hosted)
Authenticated User
Admin Features
· approve submissions
· manage products
· manage users
· post devlogs
· approve sticky notes
Tables / Data Stores
· profiles
· conversations
· messages
· gomoku_games
· week_ratings
· week_reactions
· tracker_items
· sticky_notes
· product_submissions
· products · devlog
User Features
· messaging
· gomoku game
· ratings / reactions
· tracker / sticky notes
· product submissions
read / write data
Web App
Auth Service
Database
Authenticated User
Admin
Guest

05

UML 2.5 · USE CASE DIAGRAM

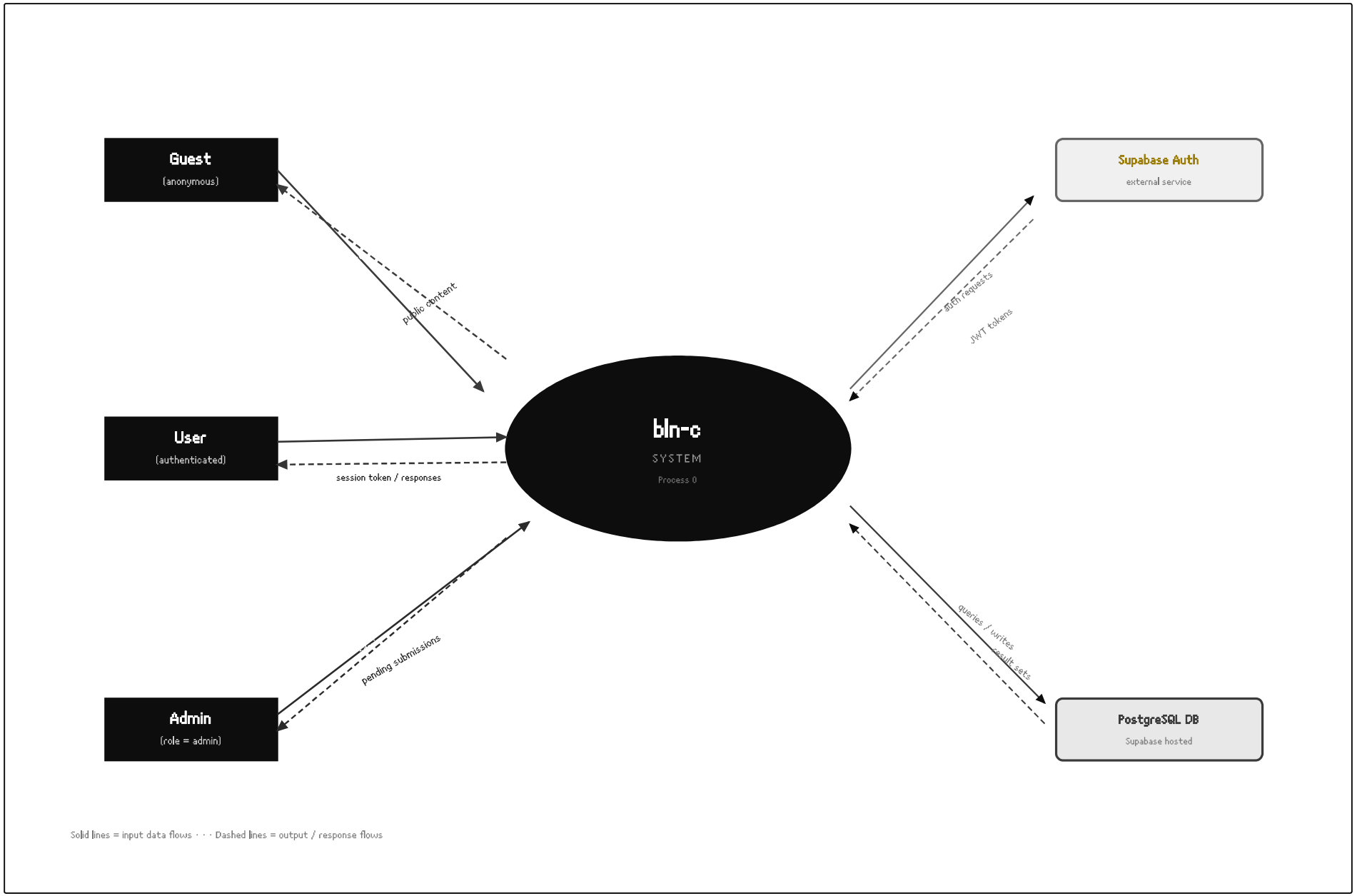
bln-c — Use Case Diagram


bln-c
Guest
Browse Public Content
Register
Login
— authenticated user actions below —
User
Manage Profile
«include»
Send / Read Messages
Create Conversations
Play Gomoku
«include»
Rate the Week
Add Reactions
Manage Tracker Items
Create Sticky Notes
Submit Products
— admin-only actions below —
Admin
Review Submissions
Approve / Reject Products
Manage Users / Roles
Manage Sticky Notes
Post Devlogs
Guest Use Cases
User Use Cases
Admin Use Cases
«include» relationship

06

DATA FLOW DIAGRAM · LEVEL 0

bln-c — Context Diagram (DFD Level 0)


Guest
(anonymous)
User
(authenticated)
Admin
(role = admin)
bln-c
SYSTEM
Process 0
Supabase Auth
external service
PostgreSQL DB
Supabase hosted
public content
session token / responses
pending submissions
auth requests
JWT tokens
queries / writes
result sets
Solid lines = input data flows · · · Dashed lines = output / response flows
External Entity
Central Process (bln-c System)
Auth Service
Database (external store)
Output / response flow

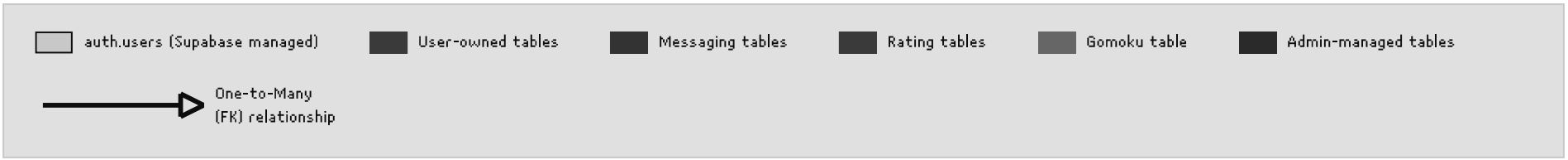
07

ENTITY-RELATIONSHIP DIAGRAM

bln-c — Database ERD (Crow's Foot Notation)

PK = Primary Key FK = Foreign Key |< = crow's foot (many end) || = one end

All UUIDs generated by gen_random_uuid(). RLS policies enforced by Supabase.


auth.users (Supabase managed)
User-owned tables
Messaging tables
Rating tables
Gomoku table
Admin-managed tables
One-to-Many
(FK) relationship

SOFTWARE REQUIREMENTS SPECIFICATION

bln-c
web app

System Documentation · Version v.6f

DOCUMENT TYPE	BACKEND	DIAGRAMS
SRS	Supabase / PostgreSQL	Pages 01–07

Table of Contents

DOCUMENT

REQUIREMENTS

DIAGRAMS

01

SOFTWARE REQUIREMENTS SPECIFICATION

Introduction

1.1 PURPOSE

This Software Requirements Specification (SRS) defines the functional requirements, external interfaces, and architectural description of **bln-c**, a full-stack web application built on a Supabase/PostgreSQL backend. The document is intended to serve as the single authoritative reference for the system's expected behaviour, linking every requirement to at least one diagram deliverable.

All seven diagram pages produced for this project are referenced and narrated here. Requirements are assigned unique identifiers (FR-01 through FR-22) so that they can be traced to specific diagrams, features, and interface contracts.

1.2 SCOPE

bln-c is a social utility web application that provides authenticated users with a suite of collaborative and personal tools: real-time messaging, a two-player Gomoku board game, a weekly mood-rating system with emoji reactions, a personal task tracker, an anonymous sticky-note board, and a community product-submission channel. An admin role controls content moderation and publishing.

The application is scoped to a single-tenant, web-delivered product. Native mobile applications, offline capability, and payment processing are explicitly out of scope for version v.6f.

1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS

SRS	Software Requirements Specification — this document.
FR	Functional Requirement — a numbered, testable statement of system behaviour.
DFD	Data Flow Diagram — shows how data moves between processes and stores.
ERD	Entity-Relationship Diagram — shows database tables and their associations.
UML	Unified Modelling Language — standard notation used for activity and use-case diagrams.
RLS	Row-Level Security — Supabase/PostgreSQL policy mechanism restricting data access per user.

JWT	JSON Web Token — signed session credential issued by Supabase Auth.
PK / FK	Primary Key / Foreign Key — relational database key types shown in the ERD (Page 07).
Guest	An unauthenticated visitor who can only access public content.
User	An authenticated account holder with standard feature access.
Admin	A user whose `profiles.role` is set to `'admin'`, granting elevated privileges.

1.4 REFERENCES

bln-c-diagrams .html	Pages 01–03: DFD Level 1, Process Flow, UML Activity Diagram.
bln-c-diagrams-p4567.html	Pages 04–07: Systems Flowchart, Use Case Diagram, DFD Level 0, ERD.
Supabase Docs	https://supabase.com/docs — Auth, RLS, Realtime, and Storage references.
Frontend Repo	Public GitHub repository containing the bln-c client-side source code. Available at: https://github.com/closuu/bln-c

1.5 DOCUMENT OVERVIEW

Section 2 provides an overall product description including stakeholder context, constraints, and assumptions. Section 3 lists all functional requirements grouped by feature area. Section 4 defines external interface contracts. Section 5 gives written narrative descriptions of all seven diagram pages. Section 6 presents a requirement traceability matrix linking each FR to the diagrams that depict it.

02

SYSTEM CONTEXT

Overall Description

2.1 PRODUCT PERSPECTIVE

bln-c is a standalone, self-contained web application. It does not form part of a larger enterprise suite. The system interfaces with two external services: **Supabase Auth** for identity management and JWT issuance, and **Supabase's hosted PostgreSQL** instance as its sole persistent data store. The client-facing layer is a browser-rendered single-page application that communicates with both services over HTTPS. Real-time features (messaging, Gomoku turn updates) are delivered via Supabase Realtime channels (WebSocket). The frontend source code is publicly available on GitHub.

> ARCHITECTURE NOTE
>
> The DFD Level 0 (Page 06) and Systems Flowchart (Page 04) together represent this product perspective. The central bln-c process in the context diagram encapsulates all application logic; the only external nodes are actors (Guest, User, Admin) and services (Supabase Auth, PostgreSQL DB).

2.2 USER CLASSES AND CHARACTERISTICS

Guest	Any visitor who has not authenticated. Can browse public-facing content (products list, devlog entries) but cannot access any interactive features. No session state is persisted server-side for guests.
Authenticated User	A registered account holder. Has full access to messaging, Gomoku, weekly ratings, tracker, sticky notes, and product submissions. All their data is isolated by RLS policies keyed to their `auth.users.id`.
Admin	An authenticated user whose `profiles.role = 'admin'`. Can review and approve/reject product submissions, promote approved submissions to the products table, post devlog entries, manage sticky-note visibility, and update user roles. Admin role is assigned directly in the database; there is no self-service promotion path.

2.3 OPERATING ENVIRONMENT

bln-c is a browser-delivered application. Users must open the site URL in a supported web browser — no installation, app store, or native binary is required. The application is designed to run correctly on both **desktop** and **mobile** form factors, adapting its layout responsively to screen width.

Desktop (any OS) | Mobile Browser | Browser Required

Supported browsers: Chrome 110+, Firefox 110+, Safari 16+, Edge 110+. JavaScript must be enabled; the application does not render meaningful content without it. There is no native iOS or Android app. The backend is hosted entirely on Supabase's cloud infrastructure; the operator does not manage any server hardware. Database, Auth, and Realtime services all reside within the same Supabase project.

> **RESPONSIVE DESIGN NOTE**
>
> The mobile layout collapses multi-column views (e.g. the messaging split-panel) into single-column stacks. The Gomoku board scales to fit the viewport width. All interactive touch targets meet a minimum size of 44×44 CSS pixels per WCAG 2.5.5 guidelines.

2.4 DESIGN AND IMPLEMENTATION CONSTRAINTS

RLS enforcement: All database tables must have Row-Level Security policies enabled. No query may bypass RLS except admin-scoped service-role calls. **UUID primary keys:** All tables use `gen_random_uuid()` as the default for primary keys; no auto-incrementing integers. **Single Supabase project:** Auth and DB must share one project so that `auth.uid()` is callable within RLS policies. **No server-side rendering required** for v.6f; the application may be delivered as a static SPA.

2.5 ASSUMPTIONS AND DEPENDENCIES

The following assumptions are made for this specification. First, email-based authentication is the sole identity provider for v.6f; OAuth social login is deferred. Second, all datetime values are stored as `timestamptz` in UTC and converted to local time client-side. Third, the Gomoku board is a fixed 15×15 grid stored as a JSON array in a single `jsonb` column; no move-history table is required for v.6f. Fourth, sticky notes require admin approval before being visible to all users, acting as a lightweight content moderation layer.

> **DEPENDENCY RISK**
>
> The system has a hard dependency on Supabase availability. Any outage in Supabase Auth or the hosted PostgreSQL instance will result in complete loss of functionality. There is no local fallback or caching layer in v.6f.

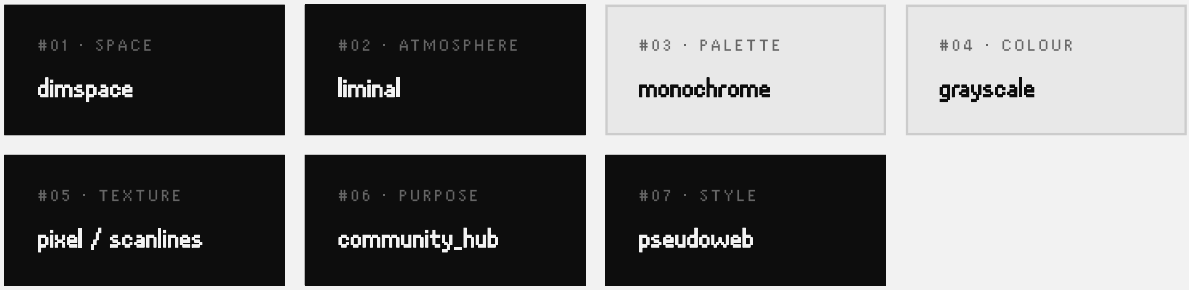
2.6 FEATURE SUMMARY

F-AUTH	Registration, login, logout, and session management via Supabase Auth.
F-PROFILE	User profile creation and editing (username, avatar, stats).
F-MSG	Direct messaging: create conversations, send/receive messages in real time.
F-GOMOKU	Two-player online Gomoku game with turn management and win detection.
F-RATE	Weekly mood ratings (1–10 scale) with optional text note and emoji reactions.
F-TRACK	Personal to-do/tracker with CRUD operations and status toggling.

F-STICKY	User-created sticky notes submitted for admin approval and public display.
F-PRODUCT	Community product submissions reviewed and approved by admins.
F-ADMIN	Admin content moderation: approve products, post devlogs, manage roles.

2.7 DESIGN INSPIRATION & AESTHETIC IDENTITY

bln-c is not designed to look like a typical modern web application. Its visual identity is intentional and draws from a curated set of aesthetic references that together define its character: a quiet, monochrome space that feels slightly removed from the present — functional yet atmospheric.



dimspace — The UI occupies space without overwhelming it. Low-contrast backgrounds, restrained use of colour, and generous whitespace give the application a dim, calm quality. Nothing competes for attention; the interface recedes so the content and the people using it can breathe.

liminal — bln-c deliberately inhabits a threshold feeling. It does not try to be a polished consumer product or a raw developer tool. Like a corridor between two rooms, it exists in-between: between public and private, between structured and freeform, between finished and ongoing. This liminality is reflected in features like the weekly rating (a pause between weeks), the sticky-note board (anonymous yet visible), and the devlog (a log of something still in progress).

monochrome & grayscale — The primary colour palette is near-black on off-white, with a single accent colour (deep red) used sparingly for destructive actions, admin indicators, and emphasis. No gradients, no hero illustrations, no photographic assets. The visual grammar is ink on paper — legible, permanent-feeling, and resistant to distraction. Colour is information, not decoration.

pixel / scanlines — The typography and grid reference an older visual language: fixed-width, monospaced type; crisp 1px rules; the occasional scanline or pixelated texture. These details signal that the interface was composed with deliberate care rather than assembled from a design kit. They evoke screens before anti-aliasing, without being retro for its own sake.

community_hub — Despite its quiet aesthetic, bln-c is fundamentally a place for people. The feature set — messaging, shared game, weekly collective mood tracking, public sticky notes, community product listings — is built around the idea of a small group of people who return regularly. It is not built for scale or virality. It is built for the specific feeling of a corner of the internet that belongs to a particular group.

pseudoweb — bln-c's interface borrows visual conventions from earlier web eras without faithfully reproducing them. It uses underlines that are not links, borders that define regions without being tables, and a typographic density that recalls the pre-CSS web — but rendered with modern layout precision. It is a simulation of an older aesthetic sensibility, rebuilt cleanly from scratch. This is the "pseudo" quality: it looks like it could have existed in 2003, but it runs in 2025.

> **DESIGN INTENT SUMMARY**
>
> These seven tags are not a mood board to be literally implemented in every component. They are a set of constraints: when a design decision arises, the question is whether the choice moves closer to or further from these qualities. A button that is too bright, a font that is too round, an animation that is too lively — these are the things the inspiration tags are meant to prevent.

03

REQUIREMENTS

Functional Requirements

Each requirement is uniquely identified, assigned a priority, and tagged with the feature area it belongs to. Priority levels: `HIGH` — must be in v.6f; `MEDIUM` — should be in v.6f, deferrable; `LOW` — nice to have.

3.1 AUTHENTICATION & SESSION MANAGEMENT

ID	PRIORITY	REQUIREMENT STATEMENT	FEATURE
FR-01	`HIGH`	The system shall allow a guest to register a new account by providing a unique email address and password. On successful registration, Supabase Auth shall create an entry in `auth.users` and trigger an upsert into `profiles`.	F-AUTH
FR-02	`HIGH`	The system shall authenticate a returning user via email and password. On success, Supabase Auth shall issue a signed JWT that the client stores and attaches to all subsequent API requests.	F-AUTH
FR-03	`HIGH`	The system shall invalidate the user's session on logout, removing the JWT from client storage and revoking the Supabase refresh token.	F-AUTH

3.2 USER PROFILE

ID	PRIORITY	REQUIREMENT STATEMENT	FEATURE
FR-04	HIGH	After registration, the system shall upsert a row in `profiles` containing the user's `id` (mirroring `auth.users.id`), email, and a default `role` of `'user'`. The `username` field must be unique across all profiles.	F-PROFILE
FR-05	MEDIUM	An authenticated user shall be able to update their `username` and `avatar_url` in the `profiles` table. RLS shall prevent any user from modifying another user's profile row.	F-PROFILE

3.3 MESSAGING

ID	PRIORITY	REQUIREMENT STATEMENT	FEATURE
FR-06	HIGH	An authenticated user shall be able to initiate a direct conversation with another user by inserting a row into `conversations` containing both `user1_id` and `user2_id`. The system shall enforce that a conversation between the same pair cannot be duplicated.	F-MSG
FR-07	HIGH	Within an existing conversation, an authenticated user shall be able to send a message by inserting a row into `messages` with the correct `conversation_id` and `sender_id`. The receiving user shall see new messages in real time via Supabase Realtime.	F-MSG

3.4 GOMOKU GAME

ID	PRIORITY	REQUIREMENT STATEMENT	FEATURE
FR-08	HIGH	An authenticated user shall be able to create or join a Gomoku game. The system shall insert a row into `gomoku_games` with `player1_id`, `player2_id`, an empty 15×15 `board` (jsonb), `current_turn` pointing to player 1, and `status = 'active'`.	F-GOMOKU
FR-09	HIGH	On a player's turn, the system shall update the `board` column in `gomoku_games` to place their stone, then flip `current_turn` to the other player. Only the player whose `id` matches `current_turn` may update the row.	F-GOMOKU
FR-10	HIGH	After each stone placement, the system shall evaluate whether the placing player has five consecutive stones in any row, column, or diagonal. If so, it shall set `winner_id` and `status = 'done'`, and increment the winner's `wins` and the loser's `losses` in `profiles`.	F-GOMOKU

3.5 WEEKLY RATINGS & REACTIONS

ID	PRIORITY	REQUIREMENT STATEMENT	FEATURE
FR-11	HIGH	An authenticated user shall be able to submit a weekly rating by inserting a row into `week_ratings` with an integer value between 1 and 10, an optional text `note`, and a `week_start` date. The system shall enforce one rating per user per week via a unique constraint on `(user_id, week_start)`.	F-RATE
FR-12	MEDIUM	An authenticated user shall be able to react to any visible weekly rating by inserting a row into `week_reactions` with the target `rating_id`, their `user_id`, and an emoji string. Multiple reactions per user per rating shall be permitted.	F-RATE

3.6 TRACKER ITEMS

ID	PRIORITY	REQUIREMENT STATEMENT	FEATURE
FR-13	HIGH	An authenticated user shall be able to create, read, update, and delete their own rows in `tracker_items`. Each item has a `title` and a `status` field constrained to the values `'todo'` or `'done'`. RLS shall isolate each user's items from all other users.	F-TRACK

3.7 STICKY NOTES

ID	PRIORITY	REQUIREMENT STATEMENT	FEATURE
FR-14	MEDIUM	An authenticated user shall be able to create a sticky note by inserting into `sticky_notes` with a `content` string and a `color` value. Newly created notes shall have `approved = false` by default and shall not be publicly visible until an admin approves them.	F-STICKY
FR-15	MEDIUM	An admin shall be able to update the `approved` column of any sticky note to `true`, making it publicly visible to all authenticated users. The admin shall also be able to delete any note.	F-STICKY / F-ADMIN

3.8 PRODUCT SUBMISSIONS

ID	PRIORITY	REQUIREMENT STATEMENT	FEATURE
FR-16	HIGH	An authenticated user shall be able to submit a product for community listing by inserting a row into `product_submissions` with a `name`, `description`, and `url`. The `approved` column defaults to `false`.	F-PRODUCT
FR-17	HIGH	An admin shall be able to review all pending `product_submissions` and either approve a submission (promoting it to the `products` table) or reject it (deleting the submission row). The `products` table shall be the sole source of truth for publicly listed products.	F-PRODUCT / F-ADMIN

3.9 ADMIN & CONTENT MANAGEMENT

ID	PRIORITY	REQUIREMENT STATEMENT	FEATURE
FR-18	HIGH	An admin shall be able to insert rows into the `devlog` table with a `title`, `content`, and `published_at` timestamp. Published devlog entries shall be visible to all visitors including unauthenticated guests.	F-ADMIN
FR-19	MEDIUM	An admin shall be able to update the `role` field of any row in `profiles` to promote or demote users between `'user'` and `'admin'`. No self-service role-change path shall exist for non-admin users.	F-ADMIN

3.10 GUEST & PUBLIC ACCESS

ID	PRIORITY	REQUIREMENT STATEMENT	FEATURE
FR-20	HIGH	An unauthenticated guest shall be able to read all rows in the `products` table and all rows in the `devlog` table without providing any credentials. All other tables shall be inaccessible to guests via RLS.	F-AUTH (access boundary)
FR-21	HIGH	The application shall redirect unauthenticated users to the login/register page if they attempt to navigate to any route requiring authentication.	F-AUTH

3.11 REAL-TIME UPDATES

ID	PRIORITY	REQUIREMENT STATEMENT	FEATURE
FR-22	MEDIUM	The system shall use Supabase Realtime subscriptions to push INSERT events on the `messages` table and UPDATE events on the `gomoku_games` table to all relevant connected clients, without requiring a manual page refresh.	F-MSG / F-GOMOKU

04

INTERFACES

External Interfaces

4.1 USER INTERFACE

UI-01	Authentication Pages (Login / Register)
DESCRIPTION	A form-based page presented to guests. Provides fields for email and password. A toggle switches between Register and Login modes. On success, the user is redirected to the dashboard. Error messages are displayed inline for invalid credentials or duplicate emails.
INPUTS	`email: string`, `password: string`, mode toggle (login | register)
OUTPUTS	JWT stored in client session; redirect to dashboard. Or inline error string.
RELATED FR	FR-01, FR-02, FR-03, FR-21

UI-02	User Dashboard
DESCRIPTION	The main authenticated view. Provides navigation to all feature modules: Messaging, Gomoku, Ratings, Tracker, Sticky Notes, and Product Submissions. Displays the user's current profile summary (username, avatar, win/loss record).
INPUTS	User session (JWT), navigation clicks
OUTPUTS	Module-specific sub-views rendered within the dashboard shell
RELATED FR	FR-04, FR-05, FR-20

UI-03	Messaging View
DESCRIPTION	A split-panel view listing active conversations on the left and the message thread on the right. New messages appear in real time. A composer field at the bottom sends messages on submit.
INPUTS	Conversation selection, message text string (max 2000 chars)
OUTPUTS	Message rows appended in real time via Supabase Realtime; INSERT to `messages`
RELATED FR	FR-06, FR-07, FR-22

UI-04	Gomoku Board View
DESCRIPTION	A 15×15 interactive grid. The current player's turn is indicated. Clicking an empty intersection places a stone. The board re-renders after each move via Realtime subscription. A win banner is shown when a winner is detected.
INPUTS	Grid cell click (row: int, col: int)
OUTPUTS	UPDATE to `gomoku_games.board` and `current_turn`; board re-render for both players
RELATED FR	FR-08, FR-09, FR-10, FR-22

UI-05	Admin Panel
DESCRIPTION	A restricted view only accessible when `profiles.role = 'admin'`. Presents tabbed sections for: pending product submissions (approve/reject actions), sticky note moderation, devlog post creation, and user role management.
INPUTS	Admin session JWT, row-level approve/reject actions, form inputs for devlog title and content
OUTPUTS	Mutations to `products`, `sticky_notes`, `devlog`, `profiles` tables
RELATED FR	FR-15, FR-17, FR-18, FR-19

4.2 SOFTWARE INTERFACES

SI-01	Supabase Auth API
INTERFACE	Supabase JavaScript client (`@supabase/supabase-js`). Methods used: `signUp()`, `signInWithPassword()`, `signOut()`, `getUser()`, `onAuthStateChange()`.
PROTOCOL	HTTPS REST to `https://<project>.supabase.co/auth/v1`. Responses include signed JWTs (RS256). Token refresh is handled automatically by the client library.
DATA FORMAT	JSON. JWT payload includes `sub` (= `auth.users.id`), `email`, `role`, `exp`.

RELATED FR	FR-01, FR-02, FR-03

SI-02	Supabase PostgreSQL (REST / PostgREST)
INTERFACE	Supabase JS client exposes a query builder over PostgREST. All table operations (SELECT, INSERT, UPDATE, DELETE) are expressed as method chains: `supabase.from('table').select()` etc.
PROTOCOL	HTTPS REST to `https://<project>.supabase.co/rest/v1`. JWT attached as `Authorization: Bearer <token>`. RLS policies evaluated server-side using `auth.uid()`.
DATA FORMAT	JSON arrays for SELECT results; JSON objects for INSERT/UPDATE. `jsonb` columns (Gomoku board) serialised as nested JSON.
RELATED FR	FR-04 through FR-20

SI-03	Supabase Realtime
INTERFACE	WebSocket-based subscription via `supabase.channel()`. The client subscribes to `postgres_changes` events on the `messages` and `gomoku_games` tables.
PROTOCOL	WebSocket (wss://) to `wss://<project>.supabase.co/realtime/v1`. Channel is authenticated using the user's JWT.
DATA FORMAT	JSON event payload: `{ eventType: 'INSERT'|'UPDATE', new: { ...row }, old: { ...row } }`.
RELATED FR	FR-22

4.3 COMMUNICATION INTERFACES

CI-01	HTTPS Transport
DESCRIPTION	All HTTP communication between the client and Supabase must use TLS 1.2 or higher. Supabase enforces this at the infrastructure level. The application must not make any plain-HTTP requests to backend services.
RELATED FR	All FRs (transport-level concern)

CI-02	WebSocket (Realtime)
DESCRIPTION	Supabase Realtime uses a persistent WebSocket connection. The client library handles reconnection with exponential back-off. If the WebSocket is unavailable, the application must degrade gracefully (features still work via polling or page refresh; no hard error shown to the user).

RELATED FR	FR-22

05

DOCUMENTATION

Written Narrative Descriptions

Each diagram produced for the bln-c project is described below in plain language. These narratives are intended to complement the visual artefacts and to explain the design intent for readers who may encounter the diagrams without a technical background.

Data Flow Diagram · Level 1

The Level 1 DFD decomposes the bln-c system into its primary data stores and the flows of data between three classes of actors and those stores. The central ellipse labelled "bln-c Application" acts as the single top-level process — it does not represent a single code module but rather the entire application boundary.

Three external entities appear on the left: **Admin**, **User**, and **Guest**. On the right, eleven data stores (D1 through D11) represent each PostgreSQL table. Stores D1–D9 are accessible to authenticated users according to RLS policies; stores D10 (`products`) and D11 (`devlog`) are admin-managed but publicly readable.

Data flows from the User entity into the application, and the application fans out to the relevant stores via a shared vertical routing bus — a design choice that keeps the diagram readable by avoiding diagonal crossing lines. A dashed flow from D10 back to the application indicates read-back of approved product data for display. The Admin entity has two additional direct flows, rendered in red, to the admin-only stores D10 and D11, emphasising the elevated privilege boundary.

The Guest entity connects via a dashed line to the application, representing read-only public browsing. This diagram is best read in conjunction with the ERD (Page 07), which shows the internal structure of each data store, and the DFD Level 0 (Page 06), which shows the same system at a higher level of abstraction.

Process Flow Diagram

The Process Flow Diagram uses four horizontal swimlanes to separate responsibilities: **Guest**, **User (Authed)**, **System/DB**, and **Admin**. Reading top-to-bottom, the diagram traces the most common user journeys through the application.

The flow begins with a Guest visiting the site and browsing public content. A decision diamond asks "Login? / Register". If the answer is No, the guest continues in a reduced capacity. If Yes, the flow crosses into the User lane, where credentials are entered. The System lane validates the credentials against Supabase Auth and upserts the profile row. On success, the User reaches the dashboard.

From the dashboard, three parallel branches radiate downward: the Messaging branch (open/send a message → conversations + messages INSERT), the Gomoku branch (create or join game → stone placement → winner check diamond → loop or end), and the Rating branch (rate the week → add reaction → week_ratings and reactions INSERT). A fourth implicit branch handles Tracker, Sticky Notes, and Product Submission in a compact lower row.

The Admin swimlane runs independently on the right, showing the admin's workflow: login, review submissions, approve or reject (decision diamond), post devlog, and manage sticky notes. This diagram directly traces FRs FR-01 through FR-19 and is the most comprehensive single view of system behaviour.

UML 2.5 Activity Diagram

The UML Activity Diagram follows the UML 2.5 standard, using formal notation: a filled circle (initial node), fork and join bars (thick horizontal bars for concurrent flow splits and merges), decision diamonds with guard conditions, and a bullseye (activity final node).

Three partitions (swimlanes) are used: **USER**, **SYSTEM**, and **ADMIN**. The flow begins in the USER partition with "Register or Login", crossing into SYSTEM for credential validation and a decision guard [valid?]. An invalid path loops back to the user for retry (dashed line). On success, the system upserts the profile and returns a session token to the USER partition.

A fork bar then splits the user's authenticated actions into concurrent branches: messaging, Gomoku (with its own internal decision for the win condition), weekly ratings, tracker management, sticky notes, and product submission. Each branch crosses into the SYSTEM partition for the corresponding database operation. A join bar at the bottom merges all branches before the user proceeds to logout, which leads to the activity final node.

This diagram is most useful for developers implementing front-end route guards and state management, as it precisely identifies which actions require an authenticated session and which system responses trigger state changes.

Systems Architecture Flowchart

The Systems Flowchart provides a high-altitude architectural view of all components and their relationships. Unlike the DFD pages, this diagram emphasises the system's physical architecture rather than data flows: it shows which external services exist, how actors connect to them, and what feature bundles each actor can access.

At the top, a **Guest** node connects to the central **bln-c Web App** node via a "visit site" edge. The Web App then branches to three downstream nodes: **Supabase Auth** (left), an **Admin** actor (centre, dashed to indicate

conditional access), and **PostgreSQL DB** (right). Each of these leads to a dashed-border feature/data box. Supabase Auth leads to an **Authenticated User** node, which leads to the User Features box. The Admin node leads to the Admin Features box. PostgreSQL DB leads to the Tables/Data Stores inventory box.

Two feedback arrows (dashed, diagonal) indicate that both the User Features and Admin Features boxes write data back into the database. This diagram is the ideal entry point for new engineers onboarding to the project, as it shows the complete system in one view without the complexity of individual data flows.

UML 2.5 Use Case Diagram

The Use Case Diagram follows UML 2.5 notation. Three actor stick figures (Guest, User, Admin) appear on the left, with the system boundary — a large dashed rectangle labelled "bln-c" — containing all use case ellipses. Use cases are organised into three horizontal bands separated by thin dashed rules, corresponding to the actor who initiates them.

The **Guest** band contains three use cases: Browse Public Content, Register, and Login. These are connected to the Guest actor by solid association lines. The **User** band contains nine use cases arranged in a 3×3 grid: the primary use cases in column 1 (Manage Profile, Play Gomoku, Manage Tracker Items) are linked to the User actor, and each includes secondary use cases in columns 2 and 3 via dashed «include» arrows (Send/Read Messages, Create Conversations; Rate the Week, Add Reactions; Create Sticky Notes, Submit Products). The **Admin** band contains five use cases: Review Submissions leads to Approve/Reject Products via an «include» arrow; Manage Sticky Notes and Post Devlogs are standalone.

This diagram is the most direct mapping to the Functional Requirements section, with every FR corresponding to at least one use case ellipse.

DFD Level 0 — Context Diagram

The Context Diagram (DFD Level 0) is the most abstract data flow view of the system. Per DFD convention, it contains exactly one process — the entire bln-c system, represented as a large filled ellipse in the centre — and all external entities and services that interact with it. There are no internal data stores at this level; they are hidden inside the process boundary.

Five external nodes surround the central process: **Guest**, **User**, and **Admin** on the left (the three actor types), and **Supabase Auth** and **PostgreSQL DB** on the right (the two external services). Each pair of adjacent entities shares two flows: a solid line representing an input (request/command) and a dashed line representing an output (response/data).

For example, the User sends "credentials / feature requests" to bln-c and receives "session token / responses" back. The Admin sends "admin commands" and receives "pending submissions". Supabase Auth exchanges "auth requests" for "JWT tokens". PostgreSQL DB receives "queries / writes" and returns "result sets".

This diagram is particularly useful for scoping discussions and for identifying all external dependencies in a single glance.

Entity-Relationship Diagram (Crow's Foot Notation)

The ERD presents all eleven database tables in Crow's Foot notation, arranged in three vertical columns. Each table is rendered as a header bar (colour-coded by functional area) plus alternating-shade field rows, each labelled with its PK/FK status and data type.

Column A (left) contains the identity and messaging tables: `auth.users` (Supabase-managed, shown in a neutral grey header), `profiles` (teal), `conversations` (violet), and `messages` (violet). **Column B** (centre) contains the game and social tables: `gomoku_games` (amber), `week_ratings` (green), `week_reactions` (green), and `tracker_items` (teal). **Column C** (right) contains the content and admin tables: `sticky_notes` (teal), `product_submissions` (crimson), `products` (crimson), and `devlog` (black).

Relationship lines use crow's foot markers: a single bar (||) at the "one" end and a three-pronged crow's foot at the "many" end. The central relationship is `profiles` as the hub entity: it has one-to-many relationships with `conversations`, `messages`, `gomoku_games`, `week_ratings`, `week_reactions`, `tracker_items`, `sticky_notes`, and `product_submissions`. The `conversations` table has a one-to-many relationship with `messages`. `week_ratings` has a one-to-many relationship with `week_reactions`. A dashed "promotes" arrow from `product_submissions` to `products` indicates the admin approval action rather than a strict FK constraint.

06

TRACEABILITY

Requirement Traceability Matrix

The matrix below links each Functional Requirement to the diagram pages that depict or support it. A ✓ indicates the diagram directly illustrates the requirement; a — means the diagram does not cover that requirement. Diagrams are referred to by their page number.

HOW TO READ THIS MATRIX

Rows = Functional Requirements (FR-01 to FR-22). Columns = Diagram pages (01–07). A checkmark means the diagram explicitly depicts or supports the requirement. Use this matrix during reviews to verify that every

> requirement has at least one diagram backing it, and during testing to identify which diagrams to reference when a test case fails.

FR ID	PG 01 DFD L1	PG 02 PROCESS	PG 03 ACTIVITY	PG 04 SYSTEMS	PG 05 USE CASE	PG 06 DFD L0	PG 07 ERD
FR-01 Register	✓	✓	✓	✓	✓	✓	—
FR-02 Login	—	✓	✓	✓	✓	✓	—
FR-03 Logout	—	—	✓	—	—	—	—
FR-04 Profile Upsert	✓	✓	✓	✓	✓	—	✓
FR-05 Profile Edit	✓	—	—	—	✓	—	✓
FR-06 Conversations	✓	✓	✓	✓	✓	—	✓
FR-07 Messages	✓	✓	✓	✓	✓	—	✓
FR-08 Gomoku Create	✓	✓	✓	✓	✓	—	✓
FR-09 Stone Placement	—	✓	✓	—	✓	—	✓
FR-10 Win Detection	—	✓	✓	—	—	—	✓
FR-11 Week Ratings	✓	✓	✓	✓	✓	—	✓
FR-12 Reactions	✓	✓	✓	✓	✓	—	✓
FR-13 Tracker CRUD	✓	✓	✓	✓	✓	—	✓
FR-14 Sticky Create	✓	✓	✓	✓	✓	—	✓
FR-15 Sticky Approve	—	✓	✓	✓	✓	—	✓
FR-16 Submit Product	✓	✓	✓	✓	✓	—	✓
FR-17 Approve Product	✓	✓	✓	✓	✓	—	✓
FR-18 Devlog Post	✓	✓	✓	✓	✓	—	✓
FR-19 Role Management	—	—	✓	✓	✓	—	✓
FR-20 Guest Read	✓	✓	—	✓	✓	✓	—
FR-21 Auth Redirect	—	✓	✓	—	—	—	—
FR-22 Realtime	—	—	—	✓	—	✓	—

6.1 DIAGRAM COVERAGE SUMMARY

DIAGRAM	PAGE	FRS COVERED	PRIMARY STRENGTH
DFD Level 1	01	FR-01,04–09,11–14,16–18,20	Shows all 11 data stores and their actor-facing flows
Process Flow	02	FR-01–02,04,06–18,20–21	Most complete step-by-step sequence coverage
UML Activity	03	FR-01–22 (all)	Formal UML, fork/join concurrency, guard conditions
Systems Flowchart	04	FR-01–02,04,06–08,11–18,20,22	Architecture overview, service relationships
Use Case	05	FR-01–02,04–09,11–19	Actor-to-feature mapping, include relationships
DFD Level 0	06	FR-01–02,20,22	Highest abstraction; external interfaces only
ERD	07	FR-04–19	Database schema, FK constraints, cardinalities

SOFTWARE REQUIREMENTS SPECIFICATION · PART 2

bln-c
web app

System Documentation · Version v.6f

DOCUMENT TYPE	CONTINUES FROM	SECTIONS
SRS · Part 2	SRS · Part 1	07–13

Table of Contents

07

QUALITY ATTRIBUTES

Non-Functional Requirements

Non-functional requirements (NFRs) define the quality attributes and operational constraints of bln-c. They specify how the system performs its functions, not what functions it performs. Each NFR is assigned a category tag and a unique identifier (NFR-xx).

> **SCOPE NOTE**
>
> NFRs apply globally to the entire system unless otherwise stated. Where a requirement conflicts with a functional requirement, the NFR takes precedence unless a documented exception exists.

7.1 PERFORMANCE

ID	CATEGORY	REQUIREMENT	PRIORITY
NFR-01	PERFORMANCE	The initial page load (first contentful paint) shall complete within 3 seconds on a standard broadband connection (≥ 10 Mbps).	HIGH
NFR-02	PERFORMANCE	Real-time message and Gomoku board updates shall be reflected in the UI within 500 ms of the triggering database event under normal load.	HIGH
NFR-03	PERFORMANCE	API read operations (fetching conversations, tracker items, product listings) shall complete within 800 ms under normal load conditions.	MEDIUM
NFR-04	PERFORMANCE	The Gomoku board shall render and accept input within 100 ms of a stone placement event to ensure a smooth gaming experience.	MEDIUM

7.2 USABILITY

ID	CATEGORY	REQUIREMENT	PRIORITY
NFR-05	USABILITY	The application shall allow users to switch between a black-on-white (default) and a white-on-black theme using a monochrome toggle control, persistent for the session. This toggle is a first-class UI feature of bln-c, reflecting the application's monochrome aesthetic identity.	HIGH
NFR-06	USABILITY	All interactive touch targets shall meet a minimum size of 44×44 CSS pixels per WCAG 2.5.5 guidelines to ensure accessibility on mobile devices.	HIGH
NFR-07	USABILITY	Error messages shall be displayed inline, adjacent to the field or action that caused the error, within 300 ms of the failure event. Generic "something went wrong" messages are not acceptable.	MEDIUM

ID	CATEGORY	REQUIREMENT	PRIORITY
NFR-08	USABILITY	The application shall be fully operable using only a keyboard (tab navigation, enter/space for actions) for all primary user flows.	MEDIUM
NFR-09	USABILITY	The application layout shall respond correctly to viewport widths from 320px (minimum mobile) to 1920px (large desktop) without horizontal scrolling or broken layouts.	HIGH

7.3 RELIABILITY & AVAILABILITY

ID	CATEGORY	REQUIREMENT	PRIORITY
NFR-10	RELIABILITY	The system's availability is directly tied to Supabase's uptime SLA. No local fallback or caching layer is implemented in v.6f. Planned maintenance windows shall be communicated to users at least 24 hours in advance via the devlog.	HIGH
NFR-11	RELIABILITY	The application shall gracefully degrade on WebSocket disconnection: existing UI state shall remain visible and a reconnection attempt shall be made automatically within 5 seconds.	MEDIUM
NFR-12	RELIABILITY	Form submissions (tracker items, sticky notes, product submissions) shall be idempotent where possible; duplicate submissions within a 2-second window shall be suppressed client-side.	LOW

7.4 MAINTAINABILITY

ID	CATEGORY	REQUIREMENT	PRIORITY
NFR-13	MAINTAINABILITY	The frontend source code shall be maintained in a public GitHub repository (`github.com/closuu/bln-c`) with commit messages that reference the feature or fix being addressed.	HIGH
NFR-14	MAINTAINABILITY	All Supabase RLS policies shall be documented in the database schema file. No undocumented policies shall be applied to any table.	MEDIUM
NFR-15	MAINTAINABILITY	UI components shall be structured so that the monochrome theme toggle can be implemented by switching a single `data-theme` attribute on the root element, with no per-component overrides required.	MEDIUM

7.5 PORTABILITY

ID	CATEGORY	REQUIREMENT	PRIORITY
NFR-16	PORTABILITY	The application shall function correctly in Chrome 110+, Firefox 110+, Safari 16+, and Edge 110+. No browser-specific APIs shall be used without a polyfill or fallback.	HIGH

ID	CATEGORY	REQUIREMENT	PRIORITY
NFR-17	PORTABILITY	The frontend shall be deployable as a static site to any static hosting provider (Vercel, Netlify, GitHub Pages) without requiring a custom server configuration.	MEDIUM

08

SECURITY

Security Requirements

Security requirements define the constraints the system must satisfy to protect user data, prevent unauthorized access, and maintain the integrity of all stored content. bln-c delegates identity and data security primarily to Supabase's managed infrastructure, but the application layer is responsible for enforcing correct policy usage.

8.1 AUTHENTICATION & SESSION SECURITY

ID	REQUIREMENT	PRIORITY
SEC-01	All authentication shall be handled by Supabase Auth. The application shall never store plaintext passwords or implement a custom credential hashing mechanism.	HIGH
SEC-02	Session tokens (JWTs) shall be stored in memory or secure HTTP-only cookies only. Storing JWTs in `localStorage` is explicitly prohibited.	HIGH
SEC-03	On logout, the client shall call `supabase.auth.signOut()` to invalidate the session server-side. A client-side redirect alone is not sufficient.	HIGH
SEC-04	Any route or API call that requires authentication shall verify the presence of a valid session token before returning data. Unauthenticated requests shall receive a 401 response, not an empty dataset.	HIGH

8.2 DATA ACCESS & ROW-LEVEL SECURITY

CRITICAL REQUIREMENT

RLS must be enabled on every table. A table without an active RLS policy is accessible to any authenticated user, which constitutes a data breach. This is the single most important security constraint in the system.

ID	REQUIREMENT	PRIORITY
SEC-05	Row-Level Security (RLS) shall be enabled on all tables in the public schema. No table shall be left with RLS disabled in the production environment.	HIGH
SEC-06	All user-scoped data policies shall use `auth.uid()` as the isolation key. Policies shall not rely on client-supplied user ID values.	HIGH
SEC-07	Admin-only operations (approve sticky notes, promote products, post devlog entries, change user roles) shall be gated by a policy that checks `profiles.role = 'admin'`. The service-role key shall never be exposed to the client.	HIGH
SEC-08	The `profiles` table shall have a policy permitting any authenticated user to read any profile (for display purposes), but write access shall be restricted to the profile owner and admins.	HIGH
SEC-09	The `tracker_items` and `conversations` / `messages` tables shall have strict owner-only read and write policies. No cross-user data access is permitted for private content.	HIGH

8.3 TRANSPORT & INFRASTRUCTURE SECURITY

ID	REQUIREMENT	PRIORITY
SEC-10	All client-to-Supabase communication shall occur over HTTPS/WSS. Plain HTTP or unencrypted WebSocket connections are not permitted.	HIGH
SEC-11	The Supabase `anon` key may be included in the client bundle as it is intended to be public-facing, but the `service_role` key shall never be present in any client-side code or public repository.	HIGH
SEC-12	User-submitted content (sticky notes, product descriptions, devlog posts) shall be treated as untrusted input. Output shall be HTML-escaped before rendering to prevent cross-site scripting (XSS).	HIGH

Database Design

bln-c uses a single Supabase-hosted PostgreSQL instance as its sole persistent store. All tables reside in the `public` schema unless otherwise noted. Supabase Auth manages the `auth.users` table; the application does not write to it directly. Primary keys use `gen_random_uuid()` throughout; no auto-incrementing integer IDs are used.

9.1 SCHEMA OVERVIEW

The database is composed of eleven tables grouped into four functional areas:

AREA	TABLES	DESCRIPTION
Identity	`auth.users`, `profiles`	Managed by Supabase Auth and the application profile layer.
Messaging	`conversations`, `messages`	Direct messaging with real-time Realtime subscriptions.
Social / Game	`gomoku_games`, `week_ratings`, `week_reactions`, `tracker_items`, `sticky_notes`	Core feature tables for the community hub functions.
Content / Admin	`product_submissions`, `products`, `devlog`	Admin-moderated content pipeline.

All relationships use UUID foreign keys. Cascading deletes are applied where child rows have no meaning without their parent (e.g. `messages` cascade-delete when a `conversation` is removed). Soft-delete patterns are not used in v.6f.

9.2 DATABASE DICTIONARY

The following tables document each column, its data type, key status, nullability, and purpose.

profiles

Extended user profile data; one row per auth.users entry

COLUMN	TYPE	KEY	NULLABLE	DESCRIPTION
id	uuid	PK, FK → AUTH.USERS	No	Matches `auth.users.id`; set by trigger on registration.
username	text		Yes	Unique display name chosen by the user. Enforced unique at DB level.
avatar_url	text		Yes	URL to the user's avatar image stored in Supabase Storage.
role	text		No	Either `'user'` (default) or `'admin'`. Set directly in DB by admin.
created_at	timestamptz		No	Timestamp of profile creation. Defaults to `now()`.

conversations

Direct message threads between two users

COLUMN	TYPE	KEY	NULLABLE	DESCRIPTION
id	uuid	PK	No	Unique conversation identifier. Generated via `gen_random_uuid()`.
user_a	uuid	FK → PROFILES	No	One participant. No directional semantics; both participants are peers.
user_b	uuid	FK → PROFILES	No	Other participant. A unique constraint on (user_a, user_b) prevents duplicate threads.
created_at	timestamptz		No	When the conversation was first created.

messages

Individual messages within a conversation

COLUMN	TYPE	KEY	NULLABLE	DESCRIPTION
id	uuid	PK	No	Unique message identifier.
conversation_id	uuid	FK → CONVERSATIONS	No	Parent conversation. Cascade-deletes messages when conversation is deleted.
sender_id	uuid	FK → PROFILES	No	The user who sent the message.
content	text		No	The message body. Plain text; no rich formatting in v.6f.
created_at	timestamptz		No	Send timestamp. Used for ordering within a conversation.

gomoku_games

Active and completed Gomoku game sessions

COLUMN	TYPE	KEY	NULLABLE	DESCRIPTION
id	uuid	PK	No	Unique game identifier.
player_black	uuid	FK → PROFILES	No	Player assigned the black stones.
player_white	uuid	FK → PROFILES	No	Player assigned the white stones.
board	jsonb		No	15×15 grid stored as a flat JSON array. Each cell: `null`, `"B"`, or `"W"`.
current_turn	uuid	FK → PROFILES	No	The player whose turn it currently is.
winner	uuid	FK → PROFILES	Yes	Null until a winner is detected. Set by the application on win check.
status	text		No	One of: `'active'`, `'completed'`, `'abandoned'`.
created_at	timestamptz		No	Game creation timestamp.

week_ratings

Weekly mood ratings submitted by users

COLUMN	TYPE	KEY	NULLABLE	DESCRIPTION
id	uuid	PK	No	Unique rating identifier.
user_id	uuid	FK → PROFILES	No	The user who submitted the rating.
week_start	date		No	ISO date of the Monday that begins the rated week. Unique per user.
score	int2		No	Integer between 1 and 10 inclusive. Enforced by a check constraint.
note	text		Yes	Optional free-text note accompanying the rating.
created_at	timestamptz		No	Submission timestamp.

tracker_items

Personal to-do items owned by individual users

COLUMN	TYPE	KEY	NULLABLE	DESCRIPTION
id	uuid	PK	No	Unique item identifier.
user_id	uuid	FK → PROFILES	No	Owner of the tracker item. Strict owner-only RLS policy.
title	text		No	Short description of the task.
done	boolean		No	Completion status. Defaults to `false`.
created_at	timestamptz		No	Creation timestamp.

sticky_notes

Anonymous notes submitted for admin approval and public display

COLUMN	TYPE	KEY	NULLABLE	DESCRIPTION
id	uuid	PK	No	Unique note identifier.
user_id	uuid	FK → PROFILES	No	Submitting user. Not exposed publicly; only visible to admins.
content	text		No	The note body. Subject to XSS escaping before render.
approved	boolean		No	Defaults to `false`. Only approved notes are visible to all users.
created_at	timestamptz		No	Submission timestamp.

product_submissions

Community product submissions pending admin review

COLUMN	TYPE	KEY	NULLABLE	DESCRIPTION
id	uuid	PK	No	Unique submission identifier.
submitted_by	uuid	FK → PROFILES	No	The user who submitted the product.
name	text		No	Product name.
description	text		Yes	Optional product description.

COLUMN	TYPE	KEY	NULLABLE	DESCRIPTION
url	text		Yes	Link to the product. Validated as a URL by the client before submission.
approved	boolean		No	Defaults to `false`. Admin sets to `true` to promote to `products`.
created_at	timestamptz		No	Submission timestamp.

products

Admin-approved products visible to all users including guests

COLUMN	TYPE	KEY	NULLABLE	DESCRIPTION
id	uuid	PK	No	Unique product identifier.
name	text		No	Product name; copied from the approved submission.
description	text		Yes	Product description.
url	text		Yes	Product link.
published_at	timestamptz		No	When the admin promoted the submission to the products table.

devlog

Admin-authored development log entries; publicly readable

COLUMN	TYPE	KEY	NULLABLE	DESCRIPTION
id	uuid	PK	No	Unique entry identifier.
author_id	uuid	FK → PROFILES	No	The admin who authored the entry. Enforced by RLS role check.
title	text		No	Entry title.
body	text		No	Entry content. Plain text in v.6f.
created_at	timestamptz		No	Post timestamp.

9.3 RLS POLICY SUMMARY

TABLE	SELECT	INSERT	UPDATE	DELETE
`profiles`	Any authenticated user	Owner only (trigger)	Owner or admin	Admin only
`conversations`	Participants only	Authenticated users	—	Participants
`messages`	Conversation participants	Sender = current user	—	Sender only
`gomoku_games`	Players in game	Authenticated users	Current turn player	—

TABLE	SELECT	INSERT	UPDATE	DELETE
week_ratings	Any authenticated user	Owner only	Owner only	Owner only
tracker_items	Owner only	Owner only	Owner only	Owner only
sticky_notes	Approved: public; Pending: owner + admin	Authenticated users	Admin only	Owner or admin
product_submissions	Owner + admin	Authenticated users	Admin only	Owner or admin
products	Public (guest + user)	Admin only	Admin only	Admin only
devlog	Public (guest + user)	Admin only	Admin only	Admin only

10

ENGINEERING PROCESS

AI & Tooling

This section documents the tools and AI-assisted workflows used during the development of bln-c v.6f. It does not describe AI features within the application itself — bln-c contains no user-facing AI features. This section covers how AI tooling was used by the developer during the build process.

10.1 AI PROMPT ENGINEERING APPROACH

AI assistance was applied selectively and purposefully during development. The approach was not to generate entire features from prompts, but to use AI as a targeted accelerant for specific tasks — particularly patch fixes, syntax lookup, and code completion during active development sessions.

> TOOLING PHILOSOPHY
>
> AI tools were used to move faster, not to replace understanding. Every AI-suggested change was reviewed, understood, and manually accepted before being committed. No generated code was pushed without the developer reading it.

Layout and diagram planning was done in **FigJam**. FigJam served as the whiteboard for mapping out page layouts, diagramming feature flows, and sketching the structure of all seven diagram pages (DFD L0/L1, Process Flow, UML Activity, Systems Flowchart, Use Case, ERD) before any code was written. FigJam was not used for design mockups or visual UI work — only for spatial layout thinking and flow mapping.

Code assistance was provided by **GitHub Copilot**, integrated directly into VS Code. Copilot was used specifically for faster patch fixes: catching small bugs, completing repetitive patterns (RLS policy blocks, Supabase query boilerplate), and suggesting corrections during active editing. Copilot's inline completions reduced friction on well-understood tasks without introducing unknown patterns.

Prompting strategy for Copilot followed a consistent pattern: the developer wrote the intent as a comment first, then let Copilot complete the implementation, then reviewed and edited the result. This comment-first approach kept the developer's intent explicit in the codebase and made Copilot's suggestions more targeted and easier to evaluate.

> WHAT AI WAS NOT USED FOR
>
> Database schema design, RLS policy logic, real-time subscription architecture, and the application's aesthetic and UX decisions were made entirely by the developer. These are judgment-intensive areas where AI suggestions were either not sought or explicitly rejected in favour of considered decisions.

10.2 TOOLS USED

◈ FigJam ◈ GitHub Copilot (VS Code) ◈ Supabase Dashboard ◈ VS Code ◈ GitHub

TOOL	ROLE	USAGE IN BLN-C
FigJam	Layout planning	Used exclusively for mapping diagram layouts and planning the seven diagram pages. Not used for UI mockups or visual design.
GitHub Copilot	Code assistance	Integrated in VS Code for inline patch fixes and boilerplate completion. Comment-first prompting strategy throughout. All suggestions manually reviewed.
Supabase Dashboard	DB management	Used to create tables, write and test RLS policies, inspect real-time logs, and manage auth configuration.
VS Code	Primary IDE	Main development environment. Extensions: ESLint, Prettier, Copilot.
GitHub	Version control	Public repository at `github.com/closuu/bln-c`. Commit history reflects all feature additions and patch fixes.

11

REQUIREMENTS

Use Case & Feature ID Register

This section catalogues all use cases and feature identifiers defined for bln-c v.6f. Use case IDs (UC-xx) describe actor-system interactions. Feature IDs (F-xxx) defined in Part 1 §2.6 are cross-referenced here against their corresponding use cases and functional requirements.

11.1 USE CASE REGISTER

UC-01	Register Account
ACTOR	Guest
FEATURE	F-AUTH
FR LINKS	FR-01
TRIGGER	Guest navigates to the register page and submits a valid email and password.
OUTCOME	A new `auth.users` entry is created and a `profiles` row is upserted. The user is redirected to the dashboard.

UC-02	Login
ACTOR	Guest
FEATURE	F-AUTH
FR LINKS	FR-02
TRIGGER	Guest submits credentials on the login form.
OUTCOME	Supabase Auth validates credentials and issues a JWT. Session is established. User is redirected to dashboard.

UC-03	Send a Message
ACTOR	Authenticated User
FEATURE	F-MSG
FR LINKS	FR-06, FR-07
TRIGGER	User opens a conversation and submits a message.

OUTCOME	Message is inserted into `messages`. Realtime subscription pushes the INSERT event to the recipient's client within 500 ms.

UC-04	**Place a Gomoku Stone**
ACTOR	Authenticated User (current turn)
FEATURE	F-GOMOKU
FR LINKS	FR-08, FR-09, FR-10
TRIGGER	User clicks an empty intersection on the 15×15 board during their turn.
OUTCOME	The `board` JSON is updated and `current_turn` is swapped. Win detection runs; if five in a row is found, `winner` is set and status becomes `'completed'`.

UC-05	**Submit Weekly Rating**
ACTOR	Authenticated User
FEATURE	F-RATE
FR LINKS	FR-11, FR-12
TRIGGER	User submits a score (1–10) and optional note for the current week.
OUTCOME	A row is inserted into `week_ratings`. Other users may react with emoji via `week_reactions`.

UC-06	**Submit Sticky Note**
ACTOR	Authenticated User
FEATURE	F-STICKY
FR LINKS	FR-14, FR-15
TRIGGER	User submits a note for anonymous display on the community board.
OUTCOME	Note is inserted with `approved = false`. Pending admin review. Once approved, the note is visible to all users anonymously.

UC-07	**Approve Product Submission**
ACTOR	Admin
FEATURE	F-PRODUCT, F-ADMIN
FR LINKS	FR-16, FR-17
TRIGGER	Admin reviews a pending `product_submissions` row and approves it.

OUTCOME	The `approved` flag is set to `true` and the submission is promoted to the `products` table, making it publicly visible.

UC-08	**Toggle Monochrome Theme**
ACTOR	Any User (Guest or Authenticated)
FEATURE	—
NFR LINKS	NFR-05, NFR-15
TRIGGER	User activates the monochrome toggle control.
OUTCOME	The root element's `data-theme` attribute is toggled between default and inverted. All colours update via CSS variables without a page reload. Theme is persisted for the session.

11.2 FEATURE ID CROSS-REFERENCE

FEATURE ID	FEATURE NAME	USE CASES	FR IDS
F-AUTH	Authentication	UC-01, UC-02	FR-01, FR-02, FR-03, FR-21
F-PROFILE	Profile	—	FR-04, FR-05
F-MSG	Messaging	UC-03	FR-06, FR-07, FR-22
F-GOMOKU	Gomoku	UC-04	FR-08, FR-09, FR-10
F-RATE	Week Ratings	UC-05	FR-11, FR-12
F-TRACK	Tracker	—	FR-13
F-STICKY	Sticky Notes	UC-06	FR-14, FR-15
F-PRODUCT	Products	UC-07	FR-16, FR-17
F-ADMIN	Admin	UC-07	FR-15, FR-17, FR-18, FR-19

12 APPENDICES

Appendices

12.A REVISION HISTORY

VERSION	DATE	CHANGES	AUTHOR
v.6f	2025	Current version. Full SRS documented across Part 1 and Part 2. All 22 functional requirements, 17 NFRs, 12 security requirements, database dictionary, and use case register completed.	closuu

12.B OUT-OF-SCOPE ITEMS

The following items are explicitly out of scope for bln-c v.6f and are not covered by any requirement in this document:

NATIVE MOBILE APPLICATIONS

No iOS or Android native app. The application is web-only and must be accessed through a supported browser.

OFFLINE CAPABILITY

No service worker, no offline cache, no progressive web app behaviour. Full Supabase connectivity is required for all functionality.

PAYMENT PROCESSING

bln-c has no paid features, subscriptions, or transactions of any kind in v.6f.

OAUTH / SOCIAL LOGIN

Only email-and-password authentication is supported. Google, GitHub, and other OAuth providers are deferred.

EMAIL NOTIFICATIONS

No transactional email is sent for new messages, game invitations, or admin actions in v.6f.

MULTI-TENANCY

bln-c is a single-tenant application. There is no concept of workspaces, organisations, or isolated instances.

AI / ML FEATURES

The application contains no user-facing AI features. AI tooling described in §10 is a developer tool only.

13

REFERENCE

Glossary, References & Definitions

13.1 GLOSSARY

Admin	A user whose `profiles.role = 'admin'`. Granted elevated privileges for content moderation, devlog posting, and role management.
anon key	The Supabase public API key included in the client bundle. Grants access only to data permitted by RLS policies for unauthenticated users.
Board state	The current configuration of stones on the 15×15 Gomoku grid, stored as a flat JSONB array of 225 cells.
Cascade delete	A database constraint that automatically deletes child rows when their parent row is deleted.
DFD	Data Flow Diagram. A diagram showing how data moves between external entities, processes, and data stores.
ERD	Entity-Relationship Diagram. Shows database tables and their associations using Crow's Foot notation.
FR	Functional Requirement. A numbered, testable statement of system behaviour. Defined in SRS Part 1 §3.
gen_random_uuid()	A PostgreSQL function that generates a Version 4 UUID. Used as the default for all primary keys in bln-c.
Guest	An unauthenticated visitor. Can read public content (products, devlog) but cannot access any interactive feature.
JWT	JSON Web Token. A signed credential issued by Supabase Auth on successful login, used to authenticate subsequent API requests.
Monochrome toggle	A UI control that switches the application between a black-on-white (default) and a white-on-black theme by toggling `data-theme` on the root element.
NFR	Non-Functional Requirement. A quality attribute constraint on the system (performance, usability, security, etc.). Defined in SRS Part 2 §7.

RLS	Row-Level Security. A PostgreSQL/Supabase policy mechanism that filters which rows a given user can read, insert, update, or delete.
Realtime	Supabase's WebSocket-based feature that pushes database change events (INSERT, UPDATE, DELETE) to subscribed clients in real time.
service_role key	A Supabase secret key that bypasses RLS entirely. Must never appear in client-side code or public repositories.
SPA	Single-Page Application. A web application that loads a single HTML document and updates the UI dynamically without full page reloads.
SRS	Software Requirements Specification. This document and its companion Part 1.
Supabase	The managed backend-as-a-service platform providing PostgreSQL, Auth, Realtime, and Storage for bln-c.
timestamptz	PostgreSQL timezone-aware timestamp type. All timestamps in bln-c are stored in UTC and converted to local time client-side.
UC	Use Case. An actor-system interaction scenario. Defined in SRS Part 2 §11.
UML	Unified Modelling Language. Standard notation used for the Activity Diagram (Page 03) and Use Case Diagram (Page 05).
UUID	Universally Unique Identifier (Version 4). Used as the primary key type for all tables in bln-c.
XSS	Cross-Site Scripting. A security vulnerability where malicious scripts are injected into web pages. Prevented in bln-c by HTML-escaping all user-submitted content before render.

13.2 EXTERNAL REFERENCES

Frontend Repo	https://github.com/closuu/bln-c — Public repository containing all client-side source code.
SRS Part 1	bln-c-srs_part1.html — Introduction, Overall Description, Functional Requirements, External Interfaces, Diagram Narratives, Traceability Matrix.
Supabase Docs	https://supabase.com/docs — Auth, RLS, Realtime, and Storage references.
IEEE 830-1998	Recommended Practice for Software Requirements Specifications. Used as structural reference for both SRS parts.
WCAG 2.1	Web Content Accessibility Guidelines. NFR-06 (touch target size) and NFR-08 (keyboard navigation) reference WCAG 2.5.5 and 2.1.1 respectively.
GitHub Copilot	https://github.com/features/copilot — AI code completion tool used in VS Code during development. See §10.
FigJam	https://www.figma.com/figjam — Online whiteboard used for diagram layout planning. See §10.

13.3 DEFINITION INDEX

The following table maps defined identifiers to the section in which they are first introduced.

PREFIX	MEANING	DEFINED IN
FR-xx	Functional Requirement	Part 1 · §3
NFR-xx	Non-Functional Requirement	Part 2 · §7
SEC-xx	Security Requirement	Part 2 · §8
UC-xx	Use Case	Part 2 · §11
F-xxx	Feature Area	Part 1 · §2.6
EI-xx	External Interface	Part 1 · §4